

C++ Programming

Day 10: STL 기초

vector, iterator, pair, algorithm, lambda

Contents

1	학습 목표	3
2	STL의 개요	3
3	vector	4
3.1	헤더와 기본 선언	4
3.2	초기화	4
3.3	원소 추가와 삭제	5
3.4	크기와 비어 있는지 확인	5
3.5	인덱스로 원소 접근	5
3.6	첫 번째와 마지막 원소	5
3.7	인덱스 반복문	6
3.8	범위 기반 반복문	6
3.9	전체 예제	6
4	iterator	7
4.1	begin과 end	7
4.2	반복자 선언	7
4.3	반복자를 이용한 순회	8
4.4	반복자로 원소 수정	8
4.5	const_iterator	8
4.6	반복자와 포인터의 유사성	8
4.7	반복자 무효화	9
4.8	전체 예제	9
5	pair	9
5.1	헤더와 기본 선언	10
5.2	초기화	10
5.3	first와 second	10
5.4	make_pair	10
5.5	pair 비교	10
5.6	vector와 pair 함께 사용하기	11
5.7	전체 예제	11
6	algorithm	12
6.1	헤더 포함	12
6.2	sort	12
6.3	find	12
6.4	count	13
6.5	reverse	13
6.6	일부 범위만 처리하기	13
6.7	pair의 기본 정렬	13
6.8	전체 예제	13
7	lambda	14
7.1	기본 문법	15
7.2	변수에 저장하기	15
7.3	매개변수와 반환값	15
7.4	값 캡처	16
7.5	참조 캡처	16
7.6	기본 캡처 방식	16

7.7	sort와 람다	17
7.8	pair를 원하는 기준으로 정렬하기	17
7.9	전체 예제	17
8	STL 요소의 연결	18
9	종합 정리	20
10	개념별 실습 완성 코드	20
10.1	vector	20
10.2	iterator	21
10.3	pair	22
10.4	algorithm	22
10.5	lambda	23
11	실습 파일 구성	24

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- STL의 의미와 컨테이너, 반복자, 알고리즘의 관계 설명하기
- `vector`를 선언하고 원소를 추가, 접근, 삭제하기
- `size()`, `empty()`, `front()`, `back()` 사용하기
- 인덱스 반복문과 범위 기반 반복문으로 `vector` 순회하기
- 반복자의 역할과 `begin()`, `end()`의 의미 설명하기
- 반복자를 역참조하여 원소를 읽고 수정하기
- `iterator`와 `const_iterator`의 차이 구분하기
- `pair`로 서로 관련된 두 값을 하나의 객체에 저장하기
- `first`, `second`, `make_pair()` 사용하기
- `sort()`, `find()`, `count()`, `reverse()` 사용하기
- 알고리즘 함수에 반복자 범위를 전달하는 방식 이해하기
- 람다 표현식의 기본 문법과 매개변수, 반환값 이해하기
- 람다를 `sort()`와 함께 사용하여 정렬 기준 정의하기
- 캡처 목록을 사용하여 외부 변수를 읽거나 수정하기

2. STL의 개요

STL(Standard Template Library)은 자주 사용하는 자료구조와 알고리즘을 템플릿으로 제공하는 C++ 표준 라이브러리의 핵심 부분이다. Day 9에서 학습한 클래스 템플릿과 함수 템플릿이 실제 라이브러리에서 어떻게 활용되는지 보여 주는 대표적인 예가 STL이다.

STL은 크게 컨테이너(container), 반복자(iterator), 알고리즘(algorithm)으로 이해할 수 있다.

구성 요소	역할	예시
컨테이너	여러 데이터를 저장하는 자료구조	<code>vector</code> , <code>list</code> , <code>map</code>
반복자	컨테이너의 원소 위치를 가리키는 객체	<code>begin()</code> , <code>end()</code>
알고리즘	반복자 범위의 데이터를 처리하는 함수	<code>sort()</code> , <code>find()</code>

예를 들어 `vector<int>`는 정수를 저장하는 컨테이너이고, `numbers.begin()`과 `numbers.end()`는 원소 범위를 나타내는 반복자를 제공한다. `sort()`는 이 반복자 범위를 받아 실제 정렬을 수행한다.

```
1 vector<int> numbers = {30, 10, 20};
2 sort(numbers.begin(), numbers.end());
```

위 문장은 다음 구조로 해석할 수 있다.

```
Container: vector<int> numbers
Range:     [numbers.begin(), numbers.end())
Algorithm: sort
```

범위의 마지막에 있는 `end()`는 마지막 원소 그 자체가 아니라 마지막 원소 다음 위치를 나타낸다. 따라서 STL의 기본 범위는 시작 위치를 포함하고 끝 위치는 포함하지 않는 반개구간 형태이다.

```
[begin, end)
begin is included
end is excluded
```

핵심 포인트

STL에서는 컨테이너가 데이터를 저장하고, 반복자가 데이터의 범위를 표현하며, 알고리즘이 그 범위를 처리한다. 이 세 요소가 서로 분리되어 있기 때문에 같은 알고리즘을 여러 컨테이너에 재사용할 수 있다.

3. vector

`vector`는 같은 자료형의 여러 값을 연속적으로 저장하는 동적 배열 컨테이너이다. 일반 배열과 달리 실행 중에 원소를 추가하거나 삭제하면서 크기를 변경할 수 있다.

3.1. 헤더와 기본 선언

`vector`를 사용하려면 `<vector>` 헤더를 포함한다.

```
1 #include <vector>
```

`vector`는 클래스 템플릿이므로 각괄호 안에 저장할 원소의 자료형을 지정한다.

```
1 vector<int> numbers;
2 vector<double> values;
3 vector<string> names;
```

각각 정수, 실수, 문자열을 저장하는 서로 다른 `vector` 타입이다.

3.2. 초기화

빈 `vector`를 만든 뒤 원소를 추가할 수도 있고, 선언과 동시에 초기값을 지정할 수도 있다.

```
1 vector<int> numbers;
2 vector<int> scores = {90, 85, 100};
```

같은 값으로 여러 원소를 초기화하는 문법도 있다.

```
1 vector<int> values(5, 10);
```

위 코드는 값이 모두 10인 정수 원소 다섯 개를 만든다.

```
10 10 10 10 10
```

다음 코드는 원소 다섯 개를 기본값으로 초기화한다.

```
1 vector<int> values(5);
```

정수의 기본값은 0이므로 다섯 원소가 모두 0이 된다.

3.3. 원소 추가와 삭제

`push_back()`은 `vector`의 끝에 새 원소를 추가한다.

```
1 vector<string> fruits;
2
3 fruits.push_back("Apple");
4 fruits.push_back("Banana");
5 fruits.push_back("Orange");
```

`pop_back()`은 마지막 원소를 제거한다.

```
1 fruits.pop_back();
```

`pop_back()`은 제거한 값을 반환하지 않는다. 또한 빈 `vector`에서 호출하면 올바른 동작이 보장되지 않으므로 필요한 경우 `empty()`로 먼저 확인한다.

```
1 if (!fruits.empty())
2 {
3     fruits.pop_back();
4 }
```

3.4. 크기와 비어 있는지 확인

`size()`는 현재 저장된 원소의 개수를 반환한다.

```
1 cout << fruits.size() << endl;
```

`empty()`는 원소가 하나도 없으면 `true`, 하나 이상 있으면 `false`를 반환한다.

```
1 if (fruits.empty())
2 {
3     cout << "Empty" << endl;
4 }
```

3.5. 인덱스로 원소 접근

배열처럼 대괄호를 사용하여 원소에 접근할 수 있다.

```
1 cout << fruits[0] << endl;
2 fruits[1] = "Mango";
```

`operator[]`는 인덱스의 유효 범위를 검사하지 않는다. 잘못된 인덱스를 사용하면 정의되지 않은 동작이 발생할 수 있다.

`at()`은 범위를 검사하는 접근 함수이다.

```
1 cout << fruits.at(0) << endl;
```

잘못된 인덱스를 전달하면 예외가 발생하므로 오류를 더 명확하게 발견할 수 있다.

3.6. 첫 번째와 마지막 원소

`front()`는 첫 번째 원소, `back()`은 마지막 원소에 접근한다.

```
1 cout << fruits.front() << endl;
2 cout << fruits.back() << endl;
```

두 함수 모두 빈 `vector`에 사용하면 안 된다.

3.7. 인덱스 반복문

`size()`를 사용하여 모든 원소를 순회할 수 있다.

```
1 for (size_t i = 0; i < fruits.size(); i++)
2 {
3     cout << fruits[i] << endl;
4 }
```

`size()`의 반환형은 부호 없는 정수 계열이므로 인덱스 변수에 `size_t`를 사용하면 자료형이 자연스럽게 맞는다.

3.8. 범위 기반 반복문

원소의 인덱스가 필요하지 않다면 범위 기반 반복문을 사용할 수 있다.

```
1 for (string fruit : fruits)
2 {
3     cout << fruit << endl;
4 }
```

위 코드는 각 원소를 `fruit`에 복사한다. 원소를 직접 수정하려면 참조를 사용한다.

```
1 for (string& fruit : fruits)
2 {
3     fruit += " Juice";
4 }
```

읽기만 하면서 불필요한 복사를 피하려면 상수 참조를 사용할 수 있다.

```
1 for (const string& fruit : fruits)
2 {
3     cout << fruit << endl;
4 }
```

3.9. 전체 예제

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4
5 using namespace std;
6
7 int main()
8 {
9     vector<string> fruits;
10
11     fruits.push_back("Apple");
12     fruits.push_back("Banana");
13     fruits.push_back("Orange");
14     fruits.push_back("Grape");
15
16     cout << "Size: " << fruits.size() << endl;
17     cout << "First: " << fruits.front() << endl;
18     cout << "Last: " << fruits.back() << endl;
19
20     for (size_t i = 0; i < fruits.size(); i++)
21     {
22         cout << fruits[i] << endl;
23     }
```

```

24     fruits.pop_back();
25
26     cout << "After pop_back" << endl;
27
28     for (const string& fruit : fruits)
29     {
30         cout << fruit << endl;
31     }
32
33     return 0;
34 }
35

```

출력 결과는 다음과 같다.

```

Size: 4
First: Apple
Last: Grape
Apple
Banana
Orange
Grape
After pop_back
Apple
Banana
Orange

```

핵심 포인트

vector는 크기가 변할 수 있는 동적 배열이다. 끝에 원소를 추가할 때는 `push_back()`, 마지막 원소를 제거할 때는 `pop_back()`, 현재 원소 수를 확인할 때는 `size()`를 사용한다.

4. iterator

반복자(iterator)는 컨테이너의 특정 원소 위치를 가리키는 객체이다. 포인터와 비슷하게 이동할 수 있고, 역참조 연산자 `*`를 사용하여 해당 위치의 원소에 접근할 수 있다.

4.1. begin과 end

`begin()`은 첫 번째 원소의 위치를 나타내는 반복자를 반환한다. `end()`는 마지막 원소 다음 위치를 나타내는 반복자를 반환한다.

```

vector:   [5] [10] [15] [20]
           ^           ^
           begin       end

```

`end()`가 가리키는 위치에는 실제 원소가 없으므로 역참조하면 안 된다.

4.2. 반복자 선언

`vector<int>`의 반복자 타입은 다음과 같이 작성한다.

```
1 vector<int>::iterator it;
```

반복자의 전체 타입이 길어질 수 있으므로 `auto`를 자주 사용한다.

```
1 auto it = numbers.begin();
```

컴파일러가 `numbers.begin()`의 반환형으로부터 반복자 타입을 추론한다.

4.3. 반복자를 이용한 순회

첫 번째 원소에서 시작하여 `end()`에 도달할 때까지 반복자를 증가시킨다.

```
1 for (auto it = numbers.begin(); it != numbers.end(); it++)
2 {
3     cout << *it << " ";
4 }
```

각 부분의 의미는 다음과 같다.

표현	의미
<code>numbers.begin()</code>	첫 번째 원소 위치
<code>numbers.end()</code>	마지막 원소 다음 위치
<code>it != numbers.end()</code>	아직 범위 안에 있는지 확인
<code>it++</code>	다음 원소 위치로 이동
<code>*it</code>	현재 위치의 원소에 접근

4.4. 반복자로 원소 수정

일반 반복자를 역참조한 결과는 실제 원소이므로 값을 수정할 수 있다.

```
1 for (auto it = numbers.begin(); it != numbers.end(); it++)
2 {
3     *it += 3;
4 }
```

각 원소에 3이 더해진다.

4.5. const_iterator

원소를 읽기만 하고 수정하지 못하게 하려면 상수 반복자를 사용한다.

```
1 vector<int>::const_iterator it;
```

`cbegin()`과 `cend()`는 상수 반복자를 반환한다.

```
1 for (auto it = numbers.cbegin(); it != numbers.cend(); it++)
2 {
3     cout << *it << " ";
4 }
```

다음과 같이 값을 수정하려고 하면 컴파일 오류가 발생한다.

```
1 for (auto it = numbers.cbegin(); it != numbers.cend(); it++)
2 {
3     *it = 100;
4 }
```

`*it`가 읽기 전용으로 취급되기 때문이다.

4.6. 반복자와 포인터의 유사성

반복자는 포인터와 완전히 같은 타입은 아니지만 사용 방식이 비슷하다.

동작	포인터	반복자
현재 값 접근	<code>*ptr</code>	<code>*it</code>
다음 위치 이동	<code>ptr++</code>	<code>it++</code>
위치 비교	<code>ptr != endPtr</code>	<code>it != container.end()</code>

반복자는 컨테이너의 구현 방식에 맞는 공통 접근 방법을 제공한다. 알고리즘 함수가 컨테이너 자체가 아니라 반복자 범위를 받는 이유도 여기에 있다.

4.7. 반복자 무효화

vector에 원소를 추가하거나 삭제하면 내부 저장 공간이 재배치될 수 있다. 이 경우 기존 반복자가 더 이상 유효하지 않을 수 있다.

```
1 auto it = numbers.begin();
2 numbers.push_back(100);
```

push_back() 과정에서 재할당이 발생했다면 it를 계속 사용하면 안 된다. 컨테이너를 변경한 뒤에는 필요한 반복자를 다시 얻는 습관이 안전하다.

4.8. 전체 예제

```
1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 int main()
7 {
8     vector<int> numbers = {5, 10, 15, 20, 25};
9
10    for (auto it = numbers.begin(); it != numbers.end(); it++)
11    {
12        cout << *it << " ";
13    }
14
15    cout << endl;
16
17    for (auto it = numbers.begin(); it != numbers.end(); it++)
18    {
19        *it += 3;
20    }
21
22    for (auto it = numbers.cbegin(); it != numbers.cend(); it++)
23    {
24        cout << *it << " ";
25    }
26
27    cout << endl;
28
29    return 0;
30 }
```

출력 결과는 다음과 같다.

```
5 10 15 20 25
8 13 18 23 28
```

핵심 포인트

반복자는 컨테이너의 원소 위치를 나타낸다. begin()부터 시작하여 end()에 도달하기 전까지 이동하고, *it로 현재 원소에 접근한다. 읽기 전용 순회에는 cbegin()과 cend()를 사용할 수 있다.

5. pair

pair는 서로 관련된 두 값을 하나의 객체에 함께 저장하는 클래스 템플릿이다. 두 값은 서로 같은 자료형일 수도 있고 서로 다른 자료형일 수도 있다.

5.1. 헤더와 기본 선언

`pair`는 `<utility>` 헤더에 정의되어 있다.

```
1 #include <utility>
```

두 타입을 각괄호 안에 순서대로 지정한다.

```
1 pair<string, int> student;
```

위 객체는 첫 번째 값으로 `string`, 두 번째 값으로 `int`를 저장한다.

5.2. 초기화

중괄호를 사용하여 선언과 동시에 두 값을 초기화할 수 있다.

```
1 pair<string, int> student = {"Alice", 95};
```

생성자 형태로도 초기화할 수 있다.

```
1 pair<string, int> student("Alice", 95);
```

5.3. first와 second

첫 번째 값은 `first`, 두 번째 값은 `second` 멤버로 접근한다.

```
1 cout << student.first << endl;
2 cout << student.second << endl;
```

두 멤버는 읽는 것뿐만 아니라 수정할 수도 있다.

```
1 student.first = "Bob";
2 student.second = 88;
```

5.4. make_pair

`make_pair()`를 사용하면 전달된 값으로부터 두 자료형을 추론하여 `pair` 객체를 생성할 수 있다.

```
1 auto student = make_pair(string("Alice"), 95);
```

문자열 리터럴을 그대로 사용하면 첫 번째 타입이 문자열 포인터 계열로 추론될 수 있다. 명확한 `pair<string, int>`를 원한다면 `string("Alice")`처럼 `string` 객체를 전달하거나 타입을 직접 선언한다.

```
1 pair<string, int> student = make_pair("Alice", 95);
```

대입 과정에서 첫 번째 값이 `string`으로 변환된다.

5.5. pair 비교

같은 타입의 `pair`끼리는 비교할 수 있다. 먼저 `first`를 비교하고, `first`가 같으면 `second`를 비교한다.

```
1 pair<int, int> a = {1, 20};
2 pair<int, int> b = {2, 10};
3
4 cout << (a < b) << endl;
```

첫 번째 값에서 $1 < 2$ 이므로 결과는 `true`이다. 이 비교 규칙은 `vector<pair<...>>`를 기본 정렬할 때도 적용된다.

5.6. vector와 pair 함께 사용하기

여러 학생의 이름과 점수를 저장하려면 `pair<string, int>`를 원소 타입으로 하는 `vector`를 만들 수 있다.

```
1 vector<pair<string, int>> students;
2
3 students.push_back({"Alice", 95});
4 students.push_back({"Bob", 88});
5 students.push_back({"Charlie", 92});
```

각 원소는 하나의 `pair` 객체이다.

```
1 for (const auto& student : students)
2 {
3     cout << student.first << " " << student.second << endl;
4 }
```

`auto`를 사용하면 긴 타입 이름을 반복해서 작성하지 않아도 된다. `const auto&`는 복사 없이 읽기 전용으로 각 원소에 접근한다.

5.7. 전체 예제

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <utility>
5
6 using namespace std;
7
8 int main()
9 {
10     pair<string, int> student = {"Alice", 95};
11
12     cout << student.first << endl;
13     cout << student.second << endl;
14
15     student.second = 98;
16
17     vector<pair<string, int>> students;
18
19     students.push_back(student);
20     students.push_back(make_pair(string("Bob"), 88));
21     students.push_back({"Charlie", 92});
22
23     for (const auto& item : students)
24     {
25         cout << item.first << ": " << item.second << endl;
26     }
27
28     return 0;
29 }
```

출력 결과는 다음과 같다.

```
Alice
95
Alice: 98
Bob: 88
Charlie: 92
```

핵심 포인트

`pair<T, U>`는 두 값을 하나로 묶는다. 첫 번째 값은 `first`, 두 번째 값은 `second`로 접근한다. `vector<pair<T, U>>` 형태를 사용하면 서로 관련된 두 값의 목록을 쉽게 관리할 수 있다.

6. algorithm

`<algorithm>` 헤더는 정렬, 탐색, 개수 계산, 순서 뒤집기 등 자주 사용하는 알고리즘 함수를 제공한다. 대부분의 알고리즘은 처리할 컨테이너 자체가 아니라 반복자로 표현한 범위를 전달받는다.

6.1. 헤더 포함

```
1 #include <algorithm>
```

6.2. sort

`sort()`는 지정한 반복자 범위의 원소를 기본적으로 오름차순 정렬한다.

```
1 vector<int> numbers = {40, 10, 30, 20};
2
3 sort(numbers.begin(), numbers.end());
```

정렬 결과는 다음과 같다.

10 20 30 40

내림차순 정렬에는 `greater<int>()`를 사용할 수 있다. 이 기능을 사용할 때는 일반적으로 `<functional>` 헤더를 포함한다.

```
1 sort(numbers.begin(), numbers.end(), greater<int>());
```

정렬 결과는 다음과 같다.

40 30 20 10

6.3. find

`find()`는 범위에서 지정한 값과 같은 첫 번째 원소를 찾는다.

```
1 auto it = find(numbers.begin(), numbers.end(), 30);
```

값을 찾으면 해당 원소를 가리키는 반복자를 반환하고, 찾지 못하면 두 번째 인수로 전달한 끝 반복자를 반환한다.

```
1 if (it != numbers.end())
2 {
3     cout << "Found: " << *it << endl;
4 }
5 else
6 {
7     cout << "Not found" << endl;
8 }
```

반환값을 바로 역참조하기 전에 반드시 `end()`와 비교해야 한다. 찾지 못한 경우 반환된 `end()`를 역참조하면 안 된다.

6.4. count

count()는 범위에서 특정 값이 등장하는 횟수를 반환한다.

```
1 vector<int> values = {1, 2, 2, 3, 2, 4};
2
3 int result = count(values.begin(), values.end(), 2);
```

result에는 3이 저장된다.

6.5. reverse

reverse()는 지정한 범위의 원소 순서를 반대로 뒤집는다.

```
1 reverse(values.begin(), values.end());
```

sort()와 달리 크기 순서로 정렬하는 것이 아니라 현재 순서를 그대로 반전한다.

6.6. 일부 범위만 처리하기

STL 알고리즘은 begin()과 end() 전체 범위뿐 아니라 일부 범위에도 적용할 수 있다.

```
1 vector<int> numbers = {50, 40, 30, 20, 10};
2
3 sort(numbers.begin(), numbers.begin() + 3);
```

처리 범위는 [numbers.begin(), numbers.begin() + 3)이므로 앞의 세 원소만 정렬된다.

30 40 50 20 10

vector의 반복자는 임의 접근 반복자이므로 begin() + n 형태의 이동이 가능하다.

6.7. pair의 기본 정렬

vector<pair<int, string>>를 기본 sort()로 정렬하면 pair의 비교 규칙이 사용된다. 먼저 first를 기준으로 정렬하고, first가 같으면 second를 기준으로 정렬한다.

```
1 vector<pair<int, string>> items = {
2     {2, "Banana"},
3     {1, "Orange"},
4     {1, "Apple"}
5 };
6
7 sort(items.begin(), items.end());
```

정렬 결과는 다음 순서가 된다.

```
1 Apple
1 Orange
2 Banana
```

6.8. 전체 예제

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 int main()
8 {
```

```

9  vector<int> numbers = {40, 10, 30, 20, 30};
10
11  sort(numbers.begin(), numbers.end());
12
13  for (int number : numbers)
14  {
15      cout << number << " ";
16  }
17
18  cout << endl;
19
20  auto it = find(numbers.begin(), numbers.end(), 30);
21
22  if (it != numbers.end())
23  {
24      cout << "Found: " << *it << endl;
25  }
26  else
27  {
28      cout << "Not found" << endl;
29  }
30
31  cout << "Count: "
32      << count(numbers.begin(), numbers.end(), 30)
33      << endl;
34
35  reverse(numbers.begin(), numbers.end());
36
37  for (int number : numbers)
38  {
39      cout << number << " ";
40  }
41
42  cout << endl;
43
44  return 0;
45 }

```

출력 결과는 다음과 같다.

```

10 20 30 30 40
Found: 30
Count: 2
40 30 30 20 10

```

핵심 포인트

STL 알고리즘은 보통 [first, last) 형태의 반복자 범위를 받는다. `sort()`는 정렬하고, `find()`는 첫 번째 일치 원소의 반복자를 반환하며, `count()`는 일치 횟수를 반환하고, `reverse()`는 현재 순서를 뒤집는다.

7. lambda

람다 표현식(lambda expression)은 이름 없는 함수를 코드 안에서 직접 만드는 문법이다. 한 번만 사용할 짧은 동작이나 STL 알고리즘에 전달할 정렬 기준을 작성할 때 특히 유용하다.

7.1. 기본 문법

람다 표현식의 기본 구조는 다음과 같다.

```
1 [capture](parameters)
2 {
3     statements
4 }
```

각 부분의 역할은 다음과 같다.

부분	의미
[capture]	외부 변수를 람다 내부에서 사용하는 방법
(parameters)	람다가 전달받는 매개변수
{ statements }	실행할 코드

가장 간단한 람다는 다음과 같다.

```
1 []()
2 {
3     cout << "Hello" << endl;
4 };
```

그러나 위 람다는 정의만 하고 호출하지 않았다. 즉시 호출하려면 뒤에 괄호를 붙인다.

```
1 []()
2 {
3     cout << "Hello" << endl;
4 }();
```

7.2. 변수에 저장하기

람다 객체를 auto 변수에 저장한 뒤 일반 함수처럼 호출할 수 있다.

```
1 auto greet = []()
2 {
3     cout << "Hello" << endl;
4 };
5
6 greet();
```

람다의 정확한 타입은 컴파일러가 생성하므로 보통 auto를 사용한다.

7.3. 매개변수와 반환값

람다는 일반 함수처럼 매개변수를 받을 수 있다.

```
1 auto add = [](int a, int b)
2 {
3     return a + b;
4 };
5
6 cout << add(10, 20) << endl;
```

반환형은 return 문으로부터 자동 추론된다. 필요하다면 후행 반환형 문법으로 직접 지정할 수 있다.

```
1 auto divide = [](double a, double b) -> double
2 {
3     return a / b;
4 };
```


7.4. 값 캡처

람다 바깥의 지역 변수를 사용하려면 캡처 목록에 지정해야 한다.

```
1 int bonus = 5;
2
3 auto addBonus = [bonus](int score)
4 {
5     return score + bonus;
6 };
```

[bonus]는 bonus를 값으로 복사하여 람다 내부에 저장한다. 이후 외부의 bonus가 바뀌어도 이미 캡처된 값에는 영향을 주지 않는다.

```
1 int bonus = 5;
2
3 auto addBonus = [bonus](int score)
4 {
5     return score + bonus;
6 };
7
8 bonus = 20;
9 cout << addBonus(90) << endl;
```

출력은 95이다. 람다를 만들 때의 값 5가 복사되었기 때문이다.

7.5. 참조 캡처

외부 변수를 직접 읽고 수정하려면 참조로 캡처한다.

```
1 int total = 0;
2
3 auto addToTotal = [&total](int value)
4 {
5     total += value;
6 };
7
8 addToTotal(10);
9 addToTotal(20);
```

람다 실행 후 외부 변수 total은 30이 된다.

7.6. 기본 캡처 방식

[=]는 사용한 외부 변수를 기본적으로 값으로 캡처한다.

```
1 auto function = [=]()
2 {
3     cout << a << " " << b << endl;
4 };
```

[&]는 사용한 외부 변수를 기본적으로 참조로 캡처한다.

```
1 auto function = [&]()
2 {
3     a++;
4     b++;
5 };
```

짧은 예제에서는 편리하지만, 어떤 변수가 어떤 방식으로 캡처되는지 불분명해질 수 있으므로 필요한 변수만 명시적으로 적는 방식이 더 읽기 쉬운 경우가 많다.

7.7. sort와 람다

sort()의 세 번째 인수에는 두 원소의 순서를 결정하는 비교 함수를 전달할 수 있다. 람다는 이 비교 기준을 코드 가까이 작성하게 한다.

```
1 vector<int> numbers = {40, 10, 30, 20};
2
3 sort(numbers.begin(), numbers.end(), [](int a, int b)
4 {
5     return a > b;
6 });
```

비교 람다가 $a > b$ 를 반환하므로 큰 값이 앞에 오는 내림차순으로 정렬된다.

비교 함수는 첫 번째 원소가 두 번째 원소보다 앞에 와야 할 때 true를 반환하도록 작성한다.

```
return a < b;    ascending order
return a > b;    descending order
```

7.8. pair를 원하는 기준으로 정렬하기

학생 이름과 점수를 저장한 pair<string, int> 목록을 점수 기준 내림차순으로 정렬하려면 second를 비교한다.

```
1 sort(students.begin(), students.end(),
2     [](const pair<string, int>& a,
3         const pair<string, int>& b)
4     {
5         return a.second > b.second;
6     });
```

두 점수가 같을 때 이름 오름차순까지 적용하려면 조건을 추가한다.

```
1 sort(students.begin(), students.end(),
2     [](const pair<string, int>& a,
3         const pair<string, int>& b)
4     {
5         if (a.second != b.second)
6         {
7             return a.second > b.second;
8         }
9
10        return a.first < b.first;
11    });
```

pair를 상수 참조로 받으면 비교 과정에서 객체를 복사하지 않고, 람다 내부에서 원소를 수정하지도 못하게 한다.

7.9. 전체 예제

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <utility>
5 #include <algorithm>
6
7 using namespace std;
8
9 int main()
10 {
11     vector<pair<string, int>> students = {
```

```

12     {"Alice", 92},
13     {"Bob", 85},
14     {"Charlie", 92},
15     {"David", 78}
16 };
17
18 sort(students.begin(), students.end(),
19      [](const pair<string, int>& a,
20         const pair<string, int>& b)
21      {
22          if (a.second != b.second)
23          {
24              return a.second > b.second;
25          }
26
27          return a.first < b.first;
28      });
29
30 for (const auto& student : students)
31 {
32     cout << student.first
33         << ": "
34         << student.second
35         << endl;
36 }
37
38 int bonus = 5;
39
40 auto addBonus = [bonus](int score)
41 {
42     return score + bonus;
43 };
44
45 cout << "Alice with bonus: "
46     << addBonus(92)
47     << endl;
48
49 return 0;
50 }

```

출력 결과는 다음과 같다.

```

Alice: 92
Charlie: 92
Bob: 85
David: 78
Alice with bonus: 97

```

핵심 포인트

람다는 이름 없는 함수 객체이다. []에는 외부 변수의 캡처 방법을 적고, ()에는 매개변수를 적는다. STL 알고리즘의 비교 기준처럼 짧고 특정 위치에서만 필요한 동작을 작성할 때 유용하다.

8. STL 요소의 연결

Day 10에서 학습한 다섯 요소는 서로 독립된 문법이 아니라 하나의 처리 흐름으로 연결된다.

```
vector<pair<string, int>>
```

```

    |
    | stores data
    v
begin() and end()
    |
    | define a range
    v
sort()
    |
    | uses a comparison rule
    v
lambda

```

다음 코드를 단계별로 해석해 보자.

```

1 vector<pair<string, int>> students = {
2     {"Alice", 92},
3     {"Bob", 85},
4     {"Charlie", 95}
5 };
6
7 sort(students.begin(), students.end(),
8     [](const auto& a, const auto& b)
9     {
10         return a.second > b.second;
11     });

```

1. vector가 여러 학생 정보를 저장한다.
2. 각 학생 정보는 이름과 점수를 묶은 pair이다.
3. begin()과 end()가 정렬할 전체 범위를 만든다.
4. sort()가 범위의 원소를 재배치한다.
5. 람다가 두 pair의 second를 비교하여 점수 내림차순 기준을 제공한다.

이 구조는 이후 더 다양한 STL 컨테이너와 알고리즘을 배울 때도 반복된다.

핵심 포인트

STL 코드를 읽을 때는 “무엇을 저장하는가”, “어떤 범위를 처리하는가”, “어떤 알고리즘을 적용하는가”, “비교 또는 처리 기준은 무엇인가”의 순서로 나누어 해석하면 이해하기 쉽다.

9. 종합 정리

핵심 포인트

- STL은 컨테이너, 반복자, 알고리즘을 중심으로 구성된다.
- `vector<T>`는 같은 타입의 여러 원소를 저장하는 동적 배열이다.
- `push_back()`은 끝에 원소를 추가하고 `pop_back()`은 마지막 원소를 제거한다.
- `size()`는 원소 수를 반환하고 `empty()`는 비어 있는지 확인한다.
- `front()`와 `back()`은 첫 번째와 마지막 원소에 접근한다.
- 범위 기반 반복문에서 참조를 사용하면 실제 원소를 수정할 수 있다.
- 반복자는 컨테이너의 원소 위치를 나타내며 포인터와 유사하게 사용한다.
- `begin()`은 첫 번째 원소, `end()`는 마지막 원소 다음 위치를 나타낸다.
- `*it`는 현재 반복자가 가리키는 원소에 접근한다.
- `cbegin()`과 `cend()`는 읽기 전용 반복자를 제공한다.
- `pair<T, U>`는 서로 관련된 두 값을 하나로 묶는다.
- `pair`의 값은 `first`와 `second`로 접근한다.
- `sort()`, `find()`, `count()`, `reverse()`는 `<algorithm>`에 정의되어 있다.
- STL 알고리즘은 일반적으로 `[first, last)` 반복자 범위를 처리한다.
- `find()`가 값을 찾지 못하면 끝 반복자를 반환하므로 역참조 전에 비교해야 한다.
- 람다는 이름 없는 함수 객체이며 `[capture](parameters) { body }` 형태로 작성한다.
- 값 캡처는 외부 값을 복사하고, 참조 캡처는 외부 변수 자체에 접근한다.
- `sort()`에 람다를 전달하면 사용자 정의 정렬 기준을 작성할 수 있다.
- `vector`, `iterator`, `pair`, `algorithm`, `lambda`는 실제 STL 코드에서 함께 연결되어 사용된다.

10. 개념별 실습 완성 코드

10.1. vector

실습

문자열을 저장하는 `vector`를 만들고 과일 네 개를 추가한다. 원소 수와 모든 원소를 출력한 뒤 마지막 원소를 제거하고 다시 출력한다.

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4
5 using namespace std;
6
```

```

7 int main()
8 {
9     vector<string> fruits;
10
11     fruits.push_back("Apple");
12     fruits.push_back("Banana");
13     fruits.push_back("Orange");
14     fruits.push_back("Grape");
15
16     cout << fruits.size() << endl;
17
18     for (size_t i = 0; i < fruits.size(); i++)
19     {
20         cout << fruits[i] << endl;
21     }
22
23     fruits.pop_back();
24
25     cout << "After pop_back" << endl;
26
27     for (const string& fruit : fruits)
28     {
29         cout << fruit << endl;
30     }
31
32     return 0;
33 }

```

10.2. iterator

실습

정수 vector의 모든 원소를 반복자로 출력한다. 일반 반복자로 각 원소에 3을 더한 뒤 상수 반복자로 결과를 출력한다.

```

1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 int main()
7 {
8     vector<int> numbers = {5, 10, 15, 20, 25};
9
10    for (auto it = numbers.begin(); it != numbers.end(); it++)
11    {
12        cout << *it << " ";
13    }
14
15    cout << endl;
16
17    for (auto it = numbers.begin(); it != numbers.end(); it++)
18    {
19        *it += 3;
20    }
21
22    for (auto it = numbers.cbegin(); it != numbers.cend(); it++)
23    {

```

```

24     cout << *it << " ";
25 }
26
27 cout << endl;
28
29 return 0;
30 }

```

10.3. pair

실습

이름과 점수를 저장하는 `pair<string, int>` 세 개를 `vector`에 넣고 모든 학생의 이름과 점수를 출력한다.

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <utility>
5
6  using namespace std;
7
8  int main()
9  {
10     vector<pair<string, int>> students;
11
12     students.push_back({"Alice", 95});
13     students.push_back({"Bob", 88});
14     students.push_back(make_pair(string("Charlie"), 92));
15
16     for (const auto& student : students)
17     {
18         cout << student.first
19             << ": "
20             << student.second
21             << endl;
22     }
23
24     return 0;
25 }

```

10.4. algorithm

실습

정수 `vector`를 오름차순 정렬한다. 값 7을 찾고, 값 3의 개수를 출력한 뒤 전체 순서를 뒤집는다.

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6
7  int main()
8  {
9     vector<int> numbers = {7, 3, 9, 3, 1, 7};

```

```

10     sort(numbers.begin(), numbers.end());
11
12     for (int number : numbers)
13     {
14         cout << number << " ";
15     }
16
17     cout << endl;
18
19     auto it = find(numbers.begin(), numbers.end(), 7);
20
21     if (it != numbers.end())
22     {
23         cout << "Found: " << *it << endl;
24     }
25     else
26     {
27         cout << "Not found" << endl;
28     }
29
30     cout << "Count of 3: "
31         << count(numbers.begin(), numbers.end(), 3)
32         << endl;
33
34     reverse(numbers.begin(), numbers.end());
35
36     for (int number : numbers)
37     {
38         cout << number << " ";
39     }
40
41     cout << endl;
42
43     return 0;
44 }
45

```

10.5. lambda

실습

학생 이름과 점수를 저장한 `vector<pair<string, int>>`를 점수 내림차순으로 정렬한다. 점수가 같으면 이름 오름차순으로 정렬한다.

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <utility>
5  #include <algorithm>
6
7  using namespace std;
8
9  int main()
10 {
11     vector<pair<string, int>> students = {
12         {"Alice", 90},
13         {"David", 85},
14         {"Bob", 90},

```



```
15     {"Charlie", 95}  
16 };  
17  
18 sort(students.begin(), students.end(),  
19      [](const pair<string, int>& a,  
20         const pair<string, int>& b)  
21      {  
22          if (a.second != b.second)  
23          {  
24              return a.second > b.second;  
25          }  
26  
27          return a.first < b.first;  
28      });  
29  
30 for (const auto& student : students)  
31 {  
32     cout << student.first  
33         << ": "  
34         << student.second  
35         << endl;  
36 }  
37  
38 return 0;  
39 }
```

11. 실습 파일 구성

실습

Day 10 파일은 다음과 같이 관리한다.

- 01_vector.cpp
- 02_iterator.cpp
- 03_pair.cpp
- 04_algorithm.cpp
- 05_lambda.cpp
- practice/p01_vector.cpp
- practice/p02_iterator.cpp
- practice/p03_pair.cpp
- practice/p04_algorithm.cpp
- practice/p05_lambda.cpp